

POLITECNICO DI TORINO



## Lab 1

### Exercise 1 & 2 - FFmpeg Audio Analysis

Music and multimedia streaming over the Internet

Academic year 2025/2026

**Full Name : MEHRAN BAYAT**

**Student Number : S320636.**

Cristina Rottondi, Leonardo Severi, Massimiliano Zanoni

Version: 2026

©2025-26

# README

Lab 1 Exercise 1 & 2: FFmpeg Audio Analysis

Audio Encoding, Decoding, Sample-wise Comparison, Error  
Metrics, and Listening Comments

CrownLabs Jupyter Environment, FFmpeg, Python, and Signal  
Analysis

Final\_Report.md

# Contents

|           |                                                          |           |
|-----------|----------------------------------------------------------|-----------|
| <b>1</b>  | <b>Title</b>                                             | <b>3</b>  |
| <b>2</b>  | <b>Abstract</b>                                          | <b>3</b>  |
| <b>3</b>  | <b>Exercise Objective</b>                                | <b>3</b>  |
| <b>4</b>  | <b>Lab Environment and Tools</b>                         | <b>4</b>  |
| 4.1       | Working Environment                                      | 4         |
| 4.2       | Tools Used                                               | 4         |
| 4.3       | Original Input File                                      | 4         |
| <b>5</b>  | <b>Part 2.1 - Using FFmpeg</b>                           | <b>5</b>  |
| 5.1       | Required Encodings                                       | 5         |
| 5.2       | Encoding Commands                                        | 5         |
| 5.3       | General FFmpeg Command Structure                         | 5         |
| 5.4       | Meaning of Each Encoding                                 | 6         |
| <b>6</b>  | <b>Part 2.2 - Analyzing Outputs</b>                      | <b>7</b>  |
| 6.1       | Conversion Back to Uncompressed WAV                      | 7         |
| 6.2       | Loading the Original and Decoded WAV Files               | 7         |
| 6.3       | Verification that All Signals Were Directly Comparable   | 8         |
| <b>7</b>  | <b>Proof that the FLAC File is Sample-wise Identical</b> | <b>8</b>  |
| <b>8</b>  | <b>Definition of the Error Signal</b>                    | <b>9</b>  |
| <b>9</b>  | <b>Mathematical Formulas Used in the Analysis</b>        | <b>9</b>  |
| 9.1       | Mean Absolute Error                                      | 9         |
| 9.2       | Mean Squared Error                                       | 9         |
| 9.3       | Signal-to-Noise Ratio                                    | 10        |
| <b>10</b> | <b>Numerical Results</b>                                 | <b>10</b> |
| <b>11</b> | <b>Detailed Discussion of the Numerical Results</b>      | <b>10</b> |
| 11.1      | File 1 - MP3 CBR 320 kbps                                | 11        |
| 11.2      | File 2 - MP3 CBR 96 kbps                                 | 11        |
| 11.3      | File 3 - MP3 VBR Maximum Quality                         | 11        |
| 11.4      | File 4 - Opus 256 kbps                                   | 11        |
| 11.5      | File 5 - Opus 48 kbps                                    | 12        |
| <b>12</b> | <b>Ranking of the Lossy Encodings</b>                    | <b>12</b> |
| <b>13</b> | <b>Graphical Representation of the SNR Results</b>       | <b>13</b> |
| 13.1      | Interpretation of the Graph                              | 13        |
| <b>14</b> | <b>Generation of the Error Files</b>                     | <b>13</b> |

|                                                                          |           |
|--------------------------------------------------------------------------|-----------|
| <b>15 Listening Analysis of the Error Files</b>                          | <b>14</b> |
| 15.1 Comments on File 1 - MP3 320 kbps                                   | 14        |
| 15.2 Comments on File 5 - Opus 48 kbps                                   | 14        |
| <b>16 Relationship Between Objective Metrics and Perceptual Analysis</b> | <b>15</b> |
| <b>17 Final Discussion</b>                                               | <b>15</b> |
| 17.1 Lossless Compression                                                | 15        |
| 17.2 High-quality Lossy Compression                                      | 16        |
| 17.3 Stronger Lossy Compression                                          | 16        |
| <b>18 Final Summary</b>                                                  | <b>17</b> |
| <b>19 Conclusion</b>                                                     | <b>17</b> |
| <b>References</b>                                                        | <b>18</b> |

# 1. Title

---

## FFmpeg Audio Compression and Audio Signal Analysis Using Lossy and Lossless Encodings

# 2. Abstract

---

In this report, I explain what I did for **Lab 1 - Exercise 1 & 2: FFmpeg Audio Analysis**. The purpose of the exercise was to use FFmpeg to encode an original WAV audio file into several compressed formats and then analyze how much each compression setting changed the audio signal.

The original audio file used in the exercise was `dogs.wav`. I encoded it into six compressed versions: three MP3 files, two Opus files, and one FLAC file. After that, I decoded all six compressed files back to uncompressed WAV format so that the decoded signals could be compared directly with the original signal.

The analysis had two important parts. First, the FLAC file had to be checked sample by sample to prove that it was identical to the original. Second, for the five lossy files, I computed the error signal, Mean Absolute Error (MAE), Mean Squared Error (MSE), and Signal-to-Noise Ratio (SNR). I also generated two audio error files and listened to them to connect the numerical results with perceptual distortion.

The final result showed that **FLAC was perfectly lossless, MP3 CBR 320 kbps was the best lossy encoding, and Opus 48 kbps produced the largest distortion** in this experiment.

# 3. Exercise Objective

---

The objective of this exercise was to understand both the practical and analytical sides of audio compression. The task was not only to run FFmpeg commands, but also to verify and measure what compression does to the samples of an audio signal.

The complete workflow can be summarized like this:

```
dogs.wav -> encode into six compressed files -> decode all compressed files  
back to WAV -> compare decoded files with the original -> compute  
objective metrics -> generate and listen to selected error files
```

For the deliverable, I had to complete the following points:

1. Encode `dogs.wav` into the six formats required by the assignment.

2. Convert each compressed file back to uncompressed WAV.
3. Prove that the FLAC-compressed version is sample-wise identical to the original file.
4. Excluding the FLAC file, compute MAE, MSE, and SNR for files 1 to 5.
5. Generate the error files for file 1 and file 5.
6. Listen to the generated error files and comment on the audible difference.

## 4. Lab Environment and Tools

---

### 4.1 Working Environment

I performed the exercise in the CrownLabs Jupyter environment. The encoding and decoding steps were performed with FFmpeg, and the signal analysis was performed in Python.

### 4.2 Tools Used

| Tool              | How I used it                                                                                                                      |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>FFmpeg</b>     | I used it to encode the original audio file into MP3, Opus, and FLAC formats, and also to decode the compressed files back to WAV. |
| <b>Python</b>     | I used it to run the numerical analysis and organize the comparison steps.                                                         |
| <b>NumPy</b>      | I used it for vectorized operations on the audio samples and the error signals.                                                    |
| <b>SciPy</b>      | I used it to read and write WAV files.                                                                                             |
| <b>Pandas</b>     | I used it to organize the numerical results in a table.                                                                            |
| <b>Matplotlib</b> | I used it in the notebook to visualize the SNR values.                                                                             |

Table 1: Main software tools used in the exercise.

### 4.3 Original Input File

The original input file for the exercise was:

```
dogs.wav
```

## 5. Part 2.1 - Using FFmpeg

---

### 5.1 Required Encodings

The original WAV file was encoded into six compressed files, exactly as required by the assignment:

1. MP3 - CBR at 320 kbps
2. MP3 - CBR at 96 kbps
3. MP3 - VBR maximum quality
4. Opus at 256 kbps
5. Opus at 48 kbps
6. FLAC with maximum compression level

### 5.2 Encoding Commands

The FFmpeg commands used for the six required encodings were:

```
ffmpeg -y -i dogs.wav -c:a libmp3lame -b:a 320k dogs_1_mp3_cbr_320.mp3
ffmpeg -y -i dogs.wav -c:a libmp3lame -b:a 96k dogs_2_mp3_cbr_096.mp3
ffmpeg -y -i dogs.wav -c:a libmp3lame -q:a 0 dogs_3_mp3_vbr_max.mp3
ffmpeg -y -i dogs.wav -c:a libopus -b:a 256k dogs_4_opus_256.opus
ffmpeg -y -i dogs.wav -c:a libopus -b:a 48k dogs_5_opus_048.opus
ffmpeg -y -i dogs.wav -c:a flac -compression_level 12 dogs_6_flac_max.flac
```

### 5.3 General FFmpeg Command Structure

Each encoding command follows the same general structure:

```
ffmpeg -y -i input_file [codec settings] output_file
```

The meaning of the most important command parts is shown below.

| Command part                       | Meaning                                                |
|------------------------------------|--------------------------------------------------------|
| <code>-y</code>                    | Overwrites the output file if it already exists.       |
| <code>-i dogs.wav</code>           | Selects <code>dogs.wav</code> as the input audio file. |
| <code>-c:a</code>                  | Selects the audio codec.                               |
| <code>-b:a</code>                  | Sets the target audio bitrate.                         |
| <code>-q:a 0</code>                | Sets the highest VBR quality for MP3.                  |
| <code>-compression_level 12</code> | Sets the maximum FLAC compression level.               |

Table 2: Explanation of the main FFmpeg options used in the encoding commands.

The required codecs were:

- `libmp3lame` for MP3,
- `libopus` for Opus,
- `flac` for FLAC.

## 5.4 Meaning of Each Encoding

**File 1 - MP3 CBR 320 kbps.** This is a high-bitrate constant-bitrate MP3 encoding. Since 320 kbps is the maximum bitrate requested in the assignment, this file is expected to preserve the original signal relatively well while still being lossy.

**File 2 - MP3 CBR 96 kbps.** This is a lower-bitrate MP3 encoding. Because the bitrate is much smaller, a stronger degradation is expected.

**File 3 - MP3 VBR maximum quality.** This is a variable-bitrate MP3 encoding in maximum-quality mode. The encoder allocates bits adaptively depending on the complexity of the signal, producing an intermediate trade-off between compression and quality.

**File 4 - Opus 256 kbps.** This is a high-bitrate Opus encoding. Since Opus is a modern perceptual codec, this version is expected to preserve the audio reasonably well.

**File 5 - Opus 48 kbps.** This is a low-bitrate Opus encoding. Because the bitrate is low, more information must be removed, so stronger distortion is expected.

**File 6 - FLAC maximum compression.** This file uses lossless compression. Even though the file is compressed, the decoded signal should remain identical to the original one sample by sample.



## 6. Part 2.2 - Analyzing Outputs

---

### 6.1 Conversion Back to Uncompressed WAV

The first required step in the analysis section was to convert the six encoded files back to uncompressed WAV. One example decoding command was:

```
ffmpeg -y -i dogs_1_mp3_cbr_320.mp3 dogs_1_mp3_cbr_320_dec.wav
```

For completeness, the full set of decoding commands was:

```
ffmpeg -y -i dogs_1_mp3_cbr_320.mp3 dogs_1_mp3_cbr_320_dec.wav
ffmpeg -y -i dogs_2_mp3_cbr_096.mp3 dogs_2_mp3_cbr_096_dec.wav
ffmpeg -y -i dogs_3_mp3_vbr_max.mp3 dogs_3_mp3_vbr_max_dec.wav
ffmpeg -y -i dogs_4_opus_256.opus dogs_4_opus_256_dec.wav
ffmpeg -y -i dogs_5_opus_048.opus dogs_5_opus_048_dec.wav
ffmpeg -y -i dogs_6_flac_max.flac dogs_6_flac_max_dec.wav
```

### 6.2 Loading the Original and Decoded WAV Files

The original file and the six decoded WAV files were loaded into the notebook for analysis. The output obtained from the notebook execution was:

```
Original: 48000 (1364219,) int16
1 MP3 CBR 320: 48000 (1364219,) int16
2 MP3 CBR 96: 48000 (1364219,) int16
3 MP3 VBR max: 48000 (1364219,) int16
4 Opus 256: 48000 (1364219,) int16
5 Opus 48: 48000 (1364219,) int16
6 FLAC max: 48000 (1364219,) int16
```

This means that:

- the sample rate of the original file was **48000 Hz**,
- the sample rate of every decoded file was also **48000 Hz**,
- each signal contained **1364219 samples**,
- all signals were stored as **int16**.

### 6.3 Verification that All Signals Were Directly Comparable

Before computing any metric, the notebook checked that all files had matching sample rates and shapes. The output was:

```
All sample rates match.  
All shapes match.
```

This verification was essential because all subsequent comparisons assumed that corresponding samples in the original and reconstructed signals referred to the same positions in time.

## 7. Proof that the FLAC File is Sample-wise Identical

The assignment explicitly required proving that the FLAC-compressed version was identical to the original sample by sample. The notebook performed these checks:

- equality of the sample rate,
- equality of the signal shape,
- direct sample-by-sample comparison using array equality,
- computation of the maximum absolute difference.

The obtained output was:

```
Same sample rate: True  
Same shape: True  
Sample-wise identical: True  
Maximum absolute difference: 0
```

This proves that if  $x$  is the original signal and  $y_6$  is the decoded FLAC signal, then:

$$y_6 = x$$

for every sample. Equivalently, the sample-wise difference is exactly zero:

$$y_6 - x = 0$$

for all sample indices. Since the maximum absolute difference was also zero, there was no numerical discrepancy between the original and the decoded FLAC version. Therefore, FLAC behaved as a **lossless codec** in this experiment.

## 8. Definition of the Error Signal

---

The assignment suggested computing the error signal first, because the same error signal was also needed for the generation of the audio error files. The analysis was performed in vectorized form in Python, without using explicit for-loops.

For each decoded lossy file  $y$ , the error signal was defined as:

$$e = y - x$$

where  $x$  is the original signal and  $y$  is the decoded compressed signal.

For the five lossy files, the error signals were:

$$e_1 = y_1 - x, \quad e_2 = y_2 - x, \quad e_3 = y_3 - x, \quad e_4 = y_4 - x, \quad e_5 = y_5 - x$$

The number of samples used in the computation was:

$$N = 1364219$$

## 9. Mathematical Formulas Used in the Analysis

---

### 9.1 Mean Absolute Error

The Mean Absolute Error (MAE) is defined as:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - x_i|$$

This metric measures the average magnitude of the absolute difference between the reconstructed signal and the original signal.

### 9.2 Mean Squared Error

The Mean Squared Error (MSE) is defined as:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - x_i)^2$$

This metric squares the error before averaging it, so larger deviations contribute more strongly.

### 9.3 Signal-to-Noise Ratio

The Signal-to-Noise Ratio (SNR) is defined as:

$$\text{SNR} = 10 \log_{10} \left( \frac{P_{\text{signal}}}{P_{\text{noise}}} \right)$$

with:

$$P_{\text{noise}} = \text{MSE}$$

and:

$$P_{\text{signal}} = \frac{1}{N} \sum_{i=1}^N x_i^2$$

The SNR measures how large the signal power is compared to the noise power introduced by the codec. A higher SNR means that the reconstructed audio is closer to the original signal.

## 10. Numerical Results

---

The computed values for files 1 to 5 were the following:

| File       | MAE        | MSE           | SNR [dB]  |
|------------|------------|---------------|-----------|
| 1_MP3_320  | 6.102702   | 76.734200     | 54.099386 |
| 2_MP3_096  | 221.709621 | 85213.442779  | 23.644194 |
| 3_MP3_VBR  | 69.738377  | 9258.827239   | 33.283715 |
| 4_Opus_256 | 67.165495  | 8370.095277   | 33.721971 |
| 5_Opus_048 | 515.549662 | 465676.278460 | 16.268434 |

Table 3: Computed MAE, MSE, and SNR values for the five lossy files.

## 11. Detailed Discussion of the Numerical Results

---

## 11.1 File 1 - MP3 CBR 320 kbps

For file 1, the measured values were:

```
MAE_1 = 6.102702  
MSE_1 = 76.734200  
SNR_1 = 54.099386 dB
```

These were the best values among the lossy encodings: the lowest MAE, the lowest MSE, and the highest SNR. This means that the decoded MP3 at 320 kbps was extremely close to the original signal. The distortion introduced by the codec was very small.

## 11.2 File 2 - MP3 CBR 96 kbps

For file 2, the measured values were:

```
MAE_2 = 221.709621  
MSE_2 = 85213.442779  
SNR_2 = 23.644194 dB
```

Compared to file 1, the degradation was much stronger. The lower bitrate forced the encoder to remove more information, and this was reflected in the substantially larger MAE and MSE and the much lower SNR.

## 11.3 File 3 - MP3 VBR Maximum Quality

For file 3, the measured values were:

```
MAE_3 = 69.738377  
MSE_3 = 9258.827239  
SNR_3 = 33.283715 dB
```

This result was clearly better than MP3 at 96 kbps and clearly worse than MP3 at 320 kbps. The variable-bitrate encoder allocated bits adaptively and produced an intermediate quality level in this experiment.

## 11.4 File 4 - Opus 256 kbps

For file 4, the measured values were:

```
MAE_4 = 67.165495  
MSE_4 = 8370.095277  
SNR_4 = 33.721971 dB
```

This file performed slightly better than file 3 because its MAE was slightly smaller, its MSE was slightly smaller, and its SNR was slightly higher. Therefore, in this experiment, Opus 256 kbps gave a marginally better reconstruction than MP3 VBR maximum quality.

### 11.5 File 5 - Opus 48 kbps

For file 5, the measured values were:

```
MAE_5 = 515.549662
MSE_5 = 465676.278460
SNR_5 = 16.268434 dB
```

This was the worst result among the lossy encodings: the largest MAE, the largest MSE, and the lowest SNR. This means that the reconstructed waveform produced by Opus 48 kbps differed substantially from the original signal.

## 12. Ranking of the Lossy Encodings

---

A quality ranking can be obtained directly from the SNR values. Since higher SNR means less distortion, the ranking was:

```
File 1 > File 4 > File 3 > File 2 > File 5
```

Explicitly, the ranking was:

1. MP3 CBR 320 kbps
2. Opus 256 kbps
3. MP3 VBR maximum quality
4. MP3 CBR 96 kbps
5. Opus 48 kbps

## 13. Graphical Representation of the SNR Results

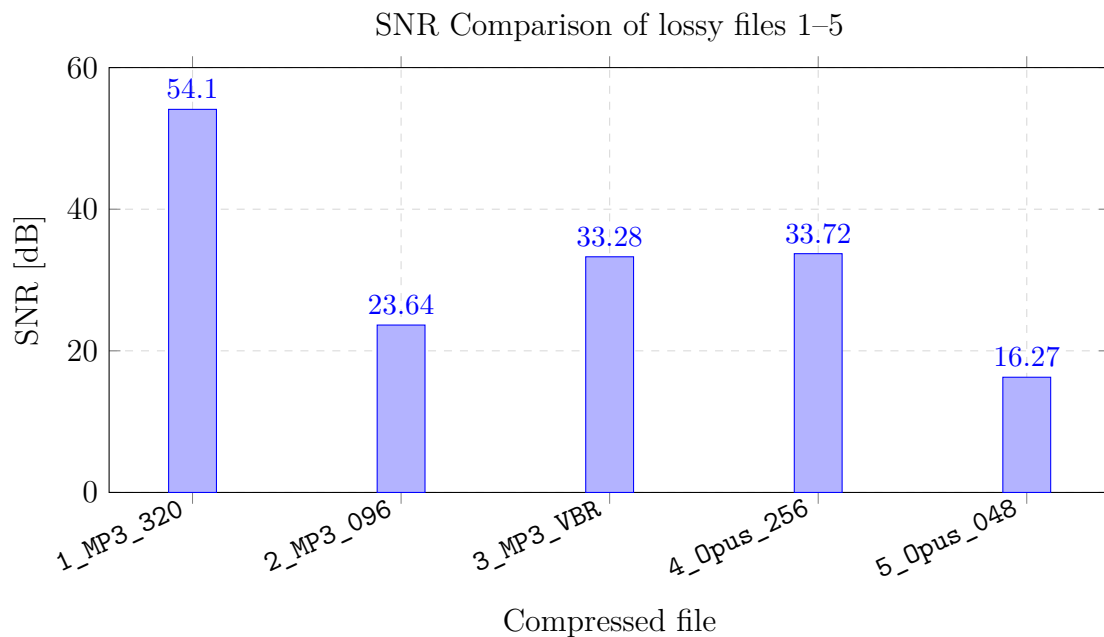


Figure 1: Bar chart of the SNR values for the five lossy compressed files.

### 13.1 Interpretation of the Graph

The graph has the five lossy encoded files on the horizontal axis and their SNR values in decibels on the vertical axis. A higher bar corresponds to a lower distortion level and therefore better reconstruction quality.

The graph shows the following:

- The bar for 1\_MP3\_320 is the tallest, reaching approximately **54.10 dB**.
- The bar for 2\_MP3\_096 is much lower, at approximately **23.64 dB**.
- The bar for 3\_MP3\_VBR is around **33.28 dB**.
- The bar for 4\_Opus\_256 is around **33.72 dB** and is slightly higher than file 3.
- The bar for 5\_Opus\_048 is the shortest, at approximately **16.27 dB**.

## 14. Generation of the Error Files

The assignment required generating the error files for file 1 and file 5. The error signals were defined as:

$$e_1 = y_1 - x$$

and:

$$e_5 = y_5 - x$$

These signals were written as WAV files:

- `error_file_1_mp3_320.wav`
- `error_file_5_opus_048.wav`

The notebook output confirmed their creation:

```
Created:  
error_file_1_mp3_320.wav  
error_file_5_opus_048.wav
```

## 15. Listening Analysis of the Error Files

---

The last required step of the assignment was to listen to the error files and comment on them.

### 15.1 Comments on File 1 - MP3 320 kbps

The comment reported in the notebook was:

```
Comments on file 1 (MP3 320 kbps): the error signal is expected to be weak  
and only slightly audible, because the compressed version is usually  
very close to the original.
```

This interpretation is consistent with the numerical results, because file 1 had the best values among the lossy encodings. Therefore, the distortion introduced by this codec setting was minimal.

### 15.2 Comments on File 5 - Opus 48 kbps

The comment reported in the notebook was:



Comments on file 5 (Opus 48 kbps): the error signal is expected to be more audible, indicating a larger distortion introduced by compression at the lower bitrate.

This is also consistent with the numerical analysis, because file 5 had the worst values among the lossy encodings. Therefore, the codec at this bitrate altered the signal much more strongly.

## 16. Relationship Between Objective Metrics and Perceptual Analysis

---

One important aspect of this exercise is that it combines objective and perceptual evaluation. The objective analysis showed that:

- file 1 had the smallest error and the highest SNR among the lossy encodings,
- file 5 had the largest error and the lowest SNR.

The listening analysis followed the same tendency: the error signal for file 1 was weak, while the error signal for file 5 was more audible. This agreement confirms that MAE, MSE, and SNR are meaningful measures of the amount of distortion introduced by compression in this experiment.

## 17. Final Discussion

---

This experiment clearly distinguishes between lossless compression and lossy compression.

### 17.1 Lossless Compression

FLAC belongs to the lossless category. The decoded FLAC file was proven to be sample-wise identical to the original:

$$y_6 = x$$

with maximum absolute difference equal to zero.

## 17.2 High-quality Lossy Compression

The following files produced relatively good reconstructions:

- MP3 CBR 320 kbps,
- MP3 VBR maximum quality,
- Opus 256 kbps.

Among these, MP3 CBR 320 kbps gave the best result in this experiment.

## 17.3 Stronger Lossy Compression

The lower-bitrate files, MP3 CBR 96 kbps and Opus 48 kbps, showed much larger errors and lower SNR values. In particular, Opus 48 kbps produced the strongest degradation among all lossy cases.

## 18. Final Summary

The exercise was completed according to all the points required by the assignment.

| Item                              | Final result                                                                    |
|-----------------------------------|---------------------------------------------------------------------------------|
| Original file                     | <code>dogs.wav</code>                                                           |
| Number of compressed files        | 6                                                                               |
| Lossy files analyzed with metrics | 5                                                                               |
| Sample rate                       | 48000 Hz                                                                        |
| Number of samples                 | 1364219                                                                         |
| Sample type                       | <code>int16</code>                                                              |
| Lossless codec result             | FLAC was sample-wise identical to the original.                                 |
| Best lossy encoding               | MP3 CBR 320 kbps.                                                               |
| Worst lossy encoding              | Opus 48 kbps.                                                                   |
| Best SNR                          | 54.099386 dB for MP3 CBR 320 kbps.                                              |
| Worst SNR                         | 16.268434 dB for Opus 48 kbps.                                                  |
| Generated error files             | <code>error_file_1_mp3_320.wav</code><br><code>error_file_5_opus_048.wav</code> |

Table 4: Final summary of the exercise results.

## 19. Conclusion

The exercise was completed according to all required points:

1. `dogs.wav` was encoded into six compressed files using FFmpeg.
2. The six compressed files were decoded back to WAV.
3. The decoded FLAC file was proven to be sample-wise identical to the original.
4. The error signals for files 1 to 5 were computed.
5. MAE, MSE, and SNR were computed using the formulas required by the assignment.
6. The error files for file 1 and file 5 were generated.

7. The error files were listened to and commented on.

The final results show that **FLAC is perfectly lossless**, **MP3 320 kbps is the best lossy encoding**, and **Opus 48 kbps is the most distorted**. The numerical metrics, the graph, and the listening comments all support the same conclusion: increasing compression generally increases distortion.

## References

---

1. Lab 1 assignment material, Music and multimedia streaming over the Internet, Politecnico di Torino, Academic year 2025/2026.
2. FFmpeg commands executed in the CrownLabs Jupyter environment.
3. Numerical analysis results obtained from the notebook execution for `dogs.wav`.